JAVA SORTING ALGORITHMS

Complete Guide: Universal Rules • Diagrams • Dry Runs • Java Code • Use Cases

Roman Hindi explanation | 1D & 2D sorting

Goal: Algorithms ratne ke bajay unka core movement samajhna, problem mein algorithm identify karna, aur Java mein khud code likhna.

Included: Bubble, Selection, Insertion, Merge, Quick, Heap, Counting, Radix Sort; complexity comparison; row-wise, column-wise aur global 2D sorting.

Contents

1. Sorting ka universal concept

2. Complexity comparison

3. Bubble Sort

4. Selection Sort

5. Insertion Sort

6. Merge Sort

7. Quick Sort

8. Heap Sort

9. Counting Sort

10. Radix Sort

11. 1D vs 2D sorting

12. 2D Java implementations

13. Algorithm selection guide

1. Sorting ka universal concept

Sorting ka matlab data ko order mein arrange karna hai: ascending (small to large) ya descending (large to small). Har algorithm correct order banata hai, lekin uska movement alag hota hai.

```
Input:      [5, 2, 8, 1, 3]
Ascending:  [1, 2, 3, 5, 8]
Descending: [8, 5, 3, 2, 1]

Ascending:  if (left > right) swap
Descending: if (left < right) swap
```

Common idea: Algorithms compare, select, shift, merge, partition, heapify, count ya digits process karke order banate hain.

Stable sorting: Equal-key records ka original relative order preserve rahe to sort stable hai. Example: same marks wale students ka pehle wala order same rahe.

2. Complexity comparison

Algorithm	Best	Average	Worst	Stable?	Extra space
Bubble	O(n)	O(n²)	O(n²)	Yes	O(1)
Selection	O(n²)	O(n²)	O(n²)	Usually no	O(1)
Insertion	O(n)	O(n²)	O(n²)	Yes	O(1)
Merge	O(n log n)	O(n log n)	O(n log n)	Yes	O(n)
Quick	O(n log n)	O(n log n)	O(n²)	No	O(log n) avg stack
Heap	O(n log n)	O(n log n)	O(n log n)	No	O(1) iterative
Counting	O(n+k)	O(n+k)	O(n+k)	Can be	O(n+k)
Radix	O(d(n+b))	O(d(n+b))	O(d(n+b))	Yes*	O(n+b)

n = number of elements; k = integer range; d = digit count; b = base (decimal mein 10). Radix stable tab hai jab har digit pass stable ho.

3. Bubble Sort

Universal rule

Adjacent (paas-paas) elements compare karo. Agar left element right se bada hai, swap. Har pass mein largest unsorted element end mein pahunchta hai.

```
Start:  [5, 1, 4, 2, 8]
```

```
Pass 1: [1, 4, 2, 5, 8]  -> 8 fix
Pass 2: [1, 2, 4, 5, 8]  -> 5 fix

[5,1] compare -> swap -> [1,5]
[5,4] compare -> swap -> [1,4,5]
```

Use: Learning aur tiny arrays. Nearly sorted array mein swap-flag se jaldi ruk sakta hai. Large arrays ke liye O(n²) slow.

```
static void bubbleSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < a.length - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

Memory line: Adjacent compare + wrong order ho to swap.

4. Selection Sort

Universal rule

Unsorted part mein minimum dhoondo aur use unsorted part ke first index par rakho. Har pass ek minimum ko final position deta hai.

```
[64, 25, 12, 22, 11]
min 11 -> [11, 25, 12, 22, 64]
min 12 -> [11, 12, 25, 22, 64]
min 22 -> [11, 12, 22, 25, 64]

Sorted | Unsorted
[11]   | [25,12,22,64]
```

Use: Simple code aur swaps kam rakhne hain. Comparisons O(n²) rehte hain; large data ke liye ideal nahi.

```
static void selectionSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        int min = i;
        for (int j = i + 1; j < a.length; j++)
            if (a[j] < a[min]) min = j;
        int t = a[i]; a[i] = a[min]; a[min] = t;
    }
}
```

Memory line: Minimum select karo + current position se swap.

5. Insertion Sort

Universal rule

Playing cards jaisa. Left side sorted maano. Next key uthao, bade elements ko right shift karo, aur key ko khaali jagah mein insert karo.

```
Start: [5, 3, 4, 1]
Key 3 -> [3, 5, 4, 1]
Key 4 -> [3, 4, 5, 1]
Key 1 -> [1, 3, 4, 5]

Sorted | Unsorted
[5]    | [3,4,1]
[3,5]  | [4,1]
```

Use: Small ya nearly sorted arrays. Best case O(n), low overhead.

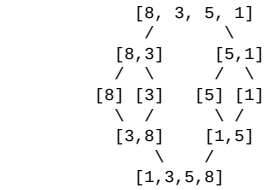
```
static void insertionSort(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

Memory line: Key uthao -> larger values shift -> key insert.

6. Merge Sort

Universal rule: Divide -> Sort -> Merge

Array ko halves mein divide karo jab tak single elements na bach jaayein. Phir do sorted halves compare karke ek sorted array banao.




```
Merge [3,8] + [1,5]:
Take 1, then 3, then 5, then 8.
```

Use: Stable O(n log n), large data, linked lists, external sorting. Array version O(n) extra memory leta hai.

```
static void mergeSort(int[] a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}
static void merge(int[] a, int l, int m, int r) {
    int[] t = new int[r - l + 1];
    int i = l, j = m + 1, k = 0;
    while (i <= m && j <= r) {
        if (a[i] <= a[j]) t[k++] = a[i++];
        else t[k++] = a[j++];
    }
    while (i <= m) t[k++] = a[i++];
    while (j <= r) t[k++] = a[j++];
    for (int x = 0; x < t.length; x++) a[l + x] = t[x];
}
```

Memory line: Divide until one element; merge two sorted halves.

7. Quick Sort

Universal rule: Pivot -> Partition -> Recurse

Pivot choose karo. Pivot se chhoti/equal values left, badi values right. Pivot final position par aata hai. Phir left aur right parts sort karo.

```
Input: [7, 2, 1, 6, 8, 5, 3, 4]
Pivot = 4
Smaller/equal | Pivot | Larger
   [2,1,3]    |   4   | [7,6,8,5]
Sort both sides -> [1,2,3,4,5,6,7,8]
```

Use: In-memory arrays mein average performance achhi. Poor pivot par worst O(n²); randomized/better pivot risk kam karta hai.

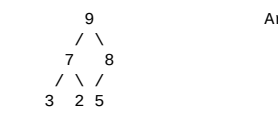
```
static void quickSort(int[] a, int low, int high) {
    if (low >= high) return;
    int p = partition(a, low, high);
    quickSort(a, low, p - 1);
    quickSort(a, p + 1, high);
}
static int partition(int[] a, int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            int t = a[i]; a[i] = a[j]; a[j] = t;
        }
    }
    int t = a[i + 1]; a[i + 1] = a[high]; a[high] = t;
    return i + 1;
}
```

Memory line: Pivot ko correct position do; left/right partitions recursively sort.

8. Heap Sort

Universal rule

Max Heap banao: parent children se bada/equal. Root maximum hota hai. Root ko last unsorted index se swap karo, heap size ghatao aur heapify karo.




```
Array: [9,7,8,3,2,5]

Root max ko end bhejo -> heapify remaining -> repeat.
Final: [2,3,5,7,8,9]
```

Use: Worst-case O(n log n), O(1) extra array space. Stable nahi hota.

```
static void heapSort(int[] a) {
    int n = a.length;
    for (int i = n / 2 - 1; i >= 0; i--) heapify(a, n, i);
    for (int end = n - 1; end > 0; end--) {
        int t = a[0]; a[0] = a[end]; a[end] = t;
        heapify(a, end, 0);
    }
}
static void heapify(int[] a, int size, int root) {
    int largest = root, left = 2*root + 1, right = 2*root + 2;
    if (left < size && a[left] > a[largest]) largest = left;
    if (right < size && a[right] > a[largest]) largest = right;
    if (largest != root) {
        int t = a[root]; a[root] = a[largest]; a[largest] = t;
        heapify(a, size, largest);
    }
}
```

Memory line: Max at root -> swap to end -> heapify remaining heap.

9. Counting Sort

Universal rule

Har integer value ki frequency count karo. Phir values ko unki frequency jitni baar output mein likho. Pairwise comparison ki zaroorat nahi.

```
Input: [4,2,2,8,3,3,1]
Value: 1 2 3 4 5 6 7 8
Count: 1 2 2 1 0 0 0 1
Output:[1,2,2,3,3,4,8]
```

Use: Integer values ka range chhota ho, jaise marks 0..100. Huge sparse range par count array wasteful.

```
// Non-negative integers only
static void countingSort(int[] a) {
    if (a.length == 0) return;
    int max = a[0];
    for (int v : a) {
        if (v < 0) throw new IllegalArgumentException("No negatives");
        if (v > max) max = v;
    }
    int[] count = new int[max + 1];
    for (int v : a) count[v]++;
    int k = 0;
    for (int v = 0; v < count.length; v++)
        while (count[v]-- > 0) a[k++] = v;
}
```

Memory line: Frequency count -> each value ko frequency jitni baar write.

10. Radix Sort

Universal rule

Numbers ko digit position ke basis par sort karo: ones, tens, hundreds... LSD Radix Sort ones digit se start karta hai. Har pass stable hona chahiye.

```
Input: [170,45,75,90,802,24,2,66]
Pass 1: ones digit par stable sort
Pass 2: tens digit par stable sort
Pass 3: hundreds digit par stable sort
Final:  [2,24,45,66,75,90,170,802]
```

Use: Non-negative integers, fixed-width IDs ya digit-based keys. Negative values ke liye sign handling add karni hogi.

```
// Non-negative integers only
static void radixSort(int[] a) {
    if (a.length == 0) return;
    int max = a[0];
    for (int v : a) {
        if (v < 0) throw new IllegalArgumentException("No negatives");
        if (v > max) max = v;
    }
    for (long exp = 1; max / exp > 0; exp *= 10)
        countByDigit(a, exp);
}
static void countByDigit(int[] a, long exp) {
    int[] out = new int[a.length], count = new int[10];
    for (int v : a) count[(int)((v / exp) % 10)]++;
    for (int i = 1; i < 10; i++) count[i] += count[i - 1];
    for (int i = a.length - 1; i >= 0; i--) {
        int d = (int)((a[i] / exp) % 10);
        out[--count[d]] = a[i];
    }
    System.arraycopy(out, 0, a, 0, a.length);
}
```

Memory line: Digit choose -> stable sort by digit -> next digit -> repeat.

11. 1D vs 2D sorting

1D: Ek straight sequence; algorithm array indices par kaam karta hai. **2D:** Rows aur columns hote hain, isliye sorting ka meaning pehle decide karna zaroori hai.

Type	Meaning	Example
Row-wise	Har row independently sort.	[[3,1],[4,2]] -> [[1,3],[2,4]]
Column-wise	Har column independently sort.	Column 0: [3,4]; column 1: [1,2]
Global	Saare cells ko ek sequence maan kar sort, phir fill.	[[3,1],[4,2]] -> [[1,2],[3,4]]

```
matrix[row][col]
row = horizontal line number
col = us row ke andar column

Row-wise:    row fix, col change
Column-wise: col fix, row change
Global:      row aur col dono traverse
```

Important: 'Sort this matrix' ambiguous ho sakta hai. Problem statement check karo: each row, each column, ya all elements together?

12. 2D Java implementations

Neeche ke methods rectangular matrix assume karte hain (har row ki length same). `import java.util.Arrays;` add karna.

A) Row-wise sorting

```
static void sortEachRow(int[][] m) {
    for (int row = 0; row < m.length; row++)
        Arrays.sort(m[row]);
}
```

Har row ek independent 1D array hai; har row par Arrays.sort().

B) Column-wise sorting (selection approach)

```
static void sortEachColumn(int[][] m) {
    int rows = m.length;
    if (rows == 0) return;
    int cols = m[0].length;

    for (int col = 0; col < cols; col++) {
        for (int i = 0; i < rows - 1; i++) {
            int minRow = i;
            for (int j = i + 1; j < rows; j++)
                if (m[j][col] < m[minRow][col]) minRow = j;

            int t = m[i][col];
            m[i][col] = m[minRow][col];
            m[minRow][col] = t;
        }
    }
}
```

Column fixed rakho; us column ke rows mein minimum find karo; current row se swap; phir next column.

C) Global sorting: flatten -> sort -> fill

```
static void sortWholeMatrix(int[][] m) {
    int rows = m.length;
    if (rows == 0) return;
    int cols = m[0].length;
    int[] a = new int[rows * cols];
    int k = 0;

    // 2D -> 1D
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            a[k++] = m[i][j];

    Arrays.sort(a);

    // Sorted 1D -> 2D
    k = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            m[i][j] = a[k++];
}
```

```
Before:        Flatten:       Sorted:       After:
[3, 1]         [3,1,4,2]      [1,2,3,4]     [1,2]
[4, 2]                                      [3,4]
```

2D universal rule: Row-wise = each row separately; Column-wise = each column separately; Global = flatten to 1D, sort, fill back.

13. Algorithm selection guide

Problem clue	Algorithm	Core reason
Adjacent compare and swap	Bubble	Adjacent exchange
Minimum choose then fix	Selection	Repeated minimum selection
Sorted left side; insert key	Insertion	Shift and insert
Half-half divide and combine	Merge	Divide and merge
Pivot and partition	Quick	Pivot establishes partitions
Max heap root	Heap	Repeated maximum extraction
Small integer range / frequency	Counting	Count each value
Ones, tens, hundreds digits	Radix	Stable digit passes

Before coding: checklist

• Ascending ya descending?

• Input 1D hai ya 2D?

• 2D mein row-wise, column-wise ya global?

• Negative values possible hain?

• n kitna hai; O(n^2) acceptable hai?

• Stable sorting required hai?

• Extra memory allowed hai?

One-line memory map: Bubble = adjacent swap; Selection = minimum select; Insertion = shift + insert; Merge = divide + merge; Quick = pivot + partition; Heap = root max + heapify; Counting = frequency; Radix = digit passes.

Practice order: Bubble -> Selection -> Insertion -> Merge -> Quick -> Heap -> Counting -> Radix. Har algorithm ko ascending, descending modification aur dry run ke saath implement karo.